

Міністерство освіти і науки України
Національний університет водного господарства та
природокористування
Кафедра автоматизації, електротехнічних та
комп'ютерно-інтегрованих технологій

04-03-334М

МЕТОДИЧНІ ВКАЗІВКИ
до виконання лабораторних робіт
з навчальної дисципліни
«Основи робототехніки»
для здобувачів вищої освіти першого (бакалаврського)
рівня за освітньо-професійною програмою
«Інтернет речей» спеціальності
121 «Інженерія програмного забезпечення»
денної форми навчання

Рекомендовано науково-
методичною радою з
якості ННІАКОТ
Протокол №5 від
28 березня 2022 р.

Рівне – 2022

Методичні вказівки до виконання лабораторних робіт з навчальної дисципліни «Основи робототехніки» для здобувачів вищої освіти першого (бакалаврського) рівня за освітньо-професійною програмою «Інтернет речей» спеціальності 121 «Інженерія програмного забезпечення» денної форми навчання [Електронне видання] / Реут Д. Т. – Рівне : НУВГП, 2022. – 50 с.

Укладач: Реут Д. Т., к.т.н., доцент кафедри автоматизації, електротехнічних та комп'ютерно-інтегрованих технологій.

Відповідальний за випуск: Древецький В. В., д.т.н., професор, завідувач кафедри автоматизації, електротехнічних та комп'ютерно-інтегрованих технологій.

Керівник групи забезпечення спеціальності 121 «Інженерія програмного забезпечення»: Жуковський В. В., к.т.н., доцент кафедри комп'ютерних наук та прикладної математики

© Реут Д. Т., 2022
© НУВГП, 2022

Зміст

Вступ	4
Лабораторна робота №1. Основи програмування в середовищі Arduino IDE. Робота з дискретними сигналами	4
Лабораторна робота №2. Організація зчитування сигналів з сенсорів	12
Лабораторна робота №3. Реалізація програмного керування колекторним двигуном постійного струму	18
Лабораторна робота №4. Дослідження роботи сервоприводів та реалізація циклограми	24
Лабораторна робота №5. Дослідження роботи маніпулятора з дистанційним управлінням	30
Лабораторна робота №6. Реалізація блокування роботи маніпулятора при виявленні перешкод	39
Лабораторна робота №7. Розробка програмного забезпечення роботизованої ділянки	42
Лабораторна робота №8. Використання комп'ютерного зору при керуванні роботом	44
Інформаційні ресурси	50

Вступ

Лабораторні роботи з навчальної дисципліни «Основи робототехніки» дають змогу освоїти основні прийоми розробки та програмування робототехнічних систем, ознайомитись з технічними засобами, які використовуються в складі робототехнічних систем на базі Arduino.

У лабораторних роботах розглянуто:

- введення та виведення сигналів за допомогою аналогових та цифрових входів та виходів мікроконтролерної плати Arduino Uno;
- визначення відстані до об'єкта;
- керування двигунами постійного струму та сервоприводами;
- керування роботизованим маніпулятором з 6 сервоприводами в режимі дистанційного й автоматичного керування в ангулярній системі координат;
- керування сервоприводом за командами системи комп'ютерного зору.

Лабораторна робота 1. Основи програмування в середовищі Arduino IDE. Вивчення роботи з вхідними та вихідними дискретними сигналами

Мета роботи: ознайомитися з апаратною та програмною частинами середовища Arduino. Вивчити основи роботи з аналоговими і дискретними сигналами в Arduino IDE.

Теоретичні відомості

Arduino – це відкрита електронна платформа, що базується на простому у використанні апаратному та програмному забезпеченні.

Апаратна частина включає мікроконтролерну плату, а програмна – середовище Arduino IDE. Платформа Arduino є відкритою, тому існує ряд аналогів Arduino-сумісних мікроконтролерних плат різного виконання.

Arduino Uno

Arduino Uno – це пристрій на основі мікроконтролера ATmega328P. До його складу входить все необхідне для зручної роботи з мікроконтролером: 14 контактів цифрових

входів/виходів (з них 6 можуть використовуватися в якості ШІМ-виходів), 6 контактів аналогових входів, кварцовий резонатор на 16 МГц, роз'єм USB, роз'єм живлення, роз'єм для внутрішньосхемного програмування (AVR ISP-6) і кнопка скидання. Для початку роботи з пристроєм достатньо просто подати живлення від AC/DC-адаптера або батарейки 9 В, або підключити його до комп'ютера за допомогою USB-кабелю.

Входи і виходи

Кожен з 14 цифрових виводів може працювати в якості входу або виходу. Рівень напруги на виводах обмежений напругою живлення 5В. Максимальний струм, який може віддавати або споживати один вивід, становить 40 мА. Усі виводи з'єднані з внутрішніми підтягуючими резисторами (за замовчуванням відключеними) номіналом 20-50 кОм. Крім цього, деякі виводи Arduino Uno можуть виконувати додаткові функції:

Послідовний інтерфейс: виводи 0 (RX) і 1 (TX). Використовуються для отримання (RX) і передачі (TX) даних по послідовному інтерфейсу. Ці виводи з'єднані з відповідними виводами ATmega8U2, що виконує роль перетворювача USB/UART (в платах-аналогах може використовуватись CH340).

Зовнішні переривання: виводи 2 і 3. Можуть служити джерелами переривань, що виникають при фронті, спаді або при низькому рівні сигналу на цих висновках.

ШІМ: виводи 3, 5, 6, 9, 10 і 11. За допомогою функції analogWrite() можуть виводити 8-бітові числа у вигляді ШІМ-сигналу (псевдоаналоговий вихід).

Інтерфейс SPI: виводи 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK). Виводи можуть задіюються для зв'язку по інтерфейсу SPI та ICSP-програмування.

Світлодіод: 13. Вбудований світлодіод, приєднаний до виводу 13.

В Arduino Uno є 6 аналогових входів (A0 - A5), на кожен з яких можна подати напругу в межах 0 – 5В та отримати аналогово-цифрове перетворення вхідного сигналу у вигляді 10-бітного числа (1024 різних значення). Верхню межу вхідної напруги можна змінити, використовуючи вивід AREF і функцію analogReference().

Таблиця 1.1

Основні характеристики Arduino Uno

Мікроконтролер	ATmega328P
Робоча напруга	5V
Вхідна напруга (рекомендована)	7-12V
Дискретні (цифрові) входи/виходи	14 (6 із них можуть використовуватися як ШІМ-виходи)
Аналогові входи	6
Максимальний струм одного вивода	40 mA
Максимальний струм вивода 3,3В	50 mA
Флеш-пам'ять	32 KB з яких 0.5 KB використовуються завантажувачем
SRAM	2 KB
EEPROM	1 KB
Тактова частота	16 MHz

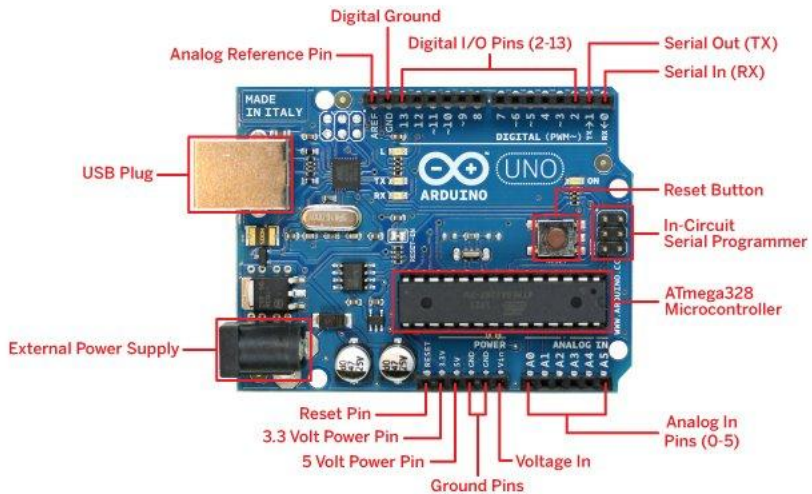


Рис. 1.1. Призначення виводів плати Arduino Uno

Arduino Mega 2560

Arduino Mega 2560 – це пристрій на основі мікроконтролера ATmega2560 (рис. 1.2).

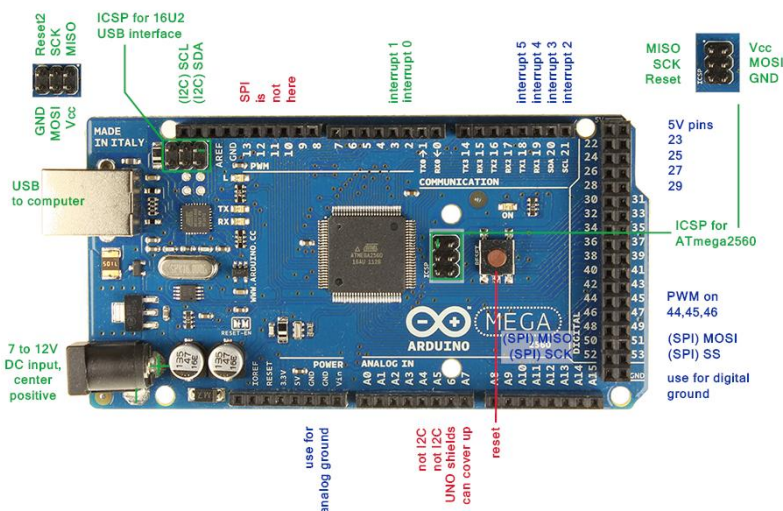


Рис. 1.2. Зовнішній вигляд та призначення виводів плати Arduino Mega 2560

Основні характеристики плати Arduino Mega 2560 наведені в табл. 1.2.

Таблиця 1.2
Основні характеристики Arduino Mega 2560

Мікроконтролер	ATmega2560
Робоча напруга	5V
Вхідна напруга (рекомендована)	7-12V
Дискретні (цифрові) входи/виходи	54 (15 із них можуть використовуватися як ШІМ-виходи)
Аналогові входи	16
Максимальний струм одного вивода	40 mA

Максимальний струм вивода 3,3В	50 mA
Флеш-пам'ять	256 KB з яких 8 KB використовуються завантажувачем
SRAM	8 KB
EEPROM	4 KB
Тактова частота	16 MHz

Програмування Arduino

Програмування мікроконтролерних плат Arduino здійснюється у вільно розповсюджуваному середовищі Arduino IDE.

Мікроконтролери в платах Arduino випускаються з прошитим завантажувачем (bootloader), що дозволяє завантажувати в мікроконтролер нові програми без необхідності використання зовнішнього програматора.

Також мікроконтролер можна прошити і через роз'єм для внутрішньосхемного програмування ICSP (In-Circuit Serial Programming), не звертаючи уваги на завантажувач.

Мова програмування Arduino – це видозмінена мова C/C++ із розширеним набором функцій.

Кожна програма для пристроїв Arduino містить дві основні функції:

1) void setup() – функція, що виконується один раз, після кожної подачі живлення або скидання плати Arduino;

2) void loop() – виконується постійно в циклі після виконання функції void setup().

Перед завантаженням програми в мікроконтролер слід спочатку вибрати тип плати Arduino та порт, до якого вона підключена.

Деякі функції для роботи з входами/виходами Arduino

pinMode(pin, value) – налаштування виводу з номером pin як дискретного входу (value=INPUT) або дискретного виходу (value=OUTPUT). Наприклад:

```
pinMode(13, OUTPUT);
```


digitalRead(pin) – функція зчитує із заданого входу значення HIGH або LOW.

digitalWrite(pin, value) – подає на цифровий вхід/вихід значення (value) HIGH або LOW;

analogRead(pin) – зчитує величину напруги з аналогового входу і видає результат у вигляді цілого числа від 0 до 1023.

План роботи

1. Ознайомитися з будовою та призначенням елементів плати Arduino Uno.
2. Ознайомитися з принципами програмування в середовищі Arduino IDE.
3. Навчитися підключати датчики з дискретними та аналоговими вихідними сигналами до Arduino та обробляти дані цих датчиків.

Порядок виконання роботи

1. Ознайомитися з теоретичними відомостями.
2. Завантажити середовище Arduino IDE.
3. Ознайомитись зі схемою підключення механічної кнопки (рис. 1.3). Зібрати її на макетній платі. Використати підтягуючий резистор на 7,5 кОм.

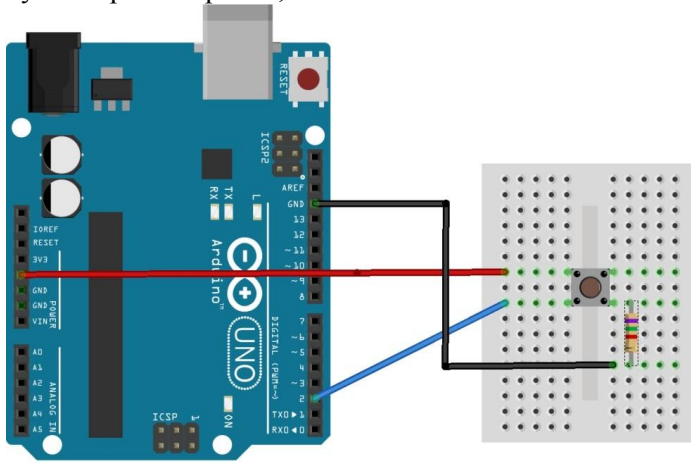


Рис. 1.3. Підключення кнопки до Arduino Uno

У середовищі Arduino IDE написати програму, що запалюватиме світлодіод при кожному натисканні кнопки. Скомпілювати програму та переконалися у відсутності помилок. Підключити мікроконтролерну плату до USB-порта комп'ютера.

У середовищі Arduino IDE вибрати тип плати та порт, до якого вона підключена. Завантажити створену програму в мікроконтролер. Переконалися в коректності роботи програми.

4. Ознайомитися зі схемою підключення ємнісної сенсорної кнопки (рис. 1.4). Зібрати її на макетній платі. Підключити сенсорну кнопку до 4 (send pin) і 6 (receive pin) дискретних виводів Arduino. Використати резистор 2,5 МОм.

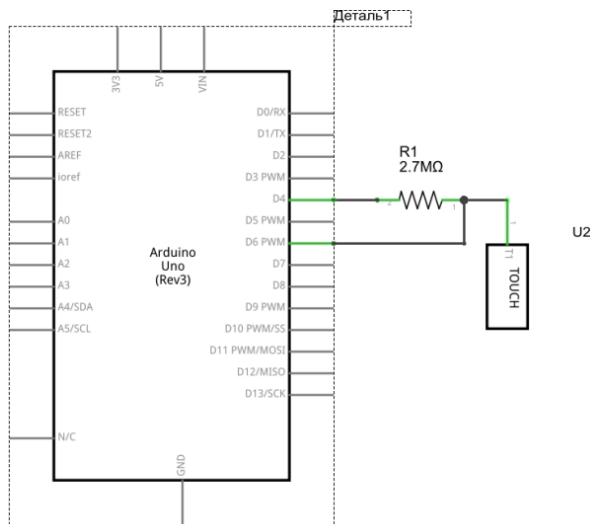


Рис. 1.4. Підключення ємнісної сенсорної кнопки до Arduino Uno

5. Завантажити бібліотеку Capacitive Sensing <http://playground.arduino.cc/Main/CapacitiveSensor?from=Main.Ca>

pSense та інсталиувати її. Написати програму, що запалюватиме світлодіод при кожному натисканні кнопки.

```
#include <CapacitiveSensor.h>

CapacitiveSensor key1 = CapacitiveSensor(4,
6); // підключена сенсорна кнопка між 4-м і 6-м
пінами

int range = 100; // порогове значення, що
визначає натискання кнопки. Підбирається
експериментально

byte button; // позначення натиснутих кнопок
void setup()
{
    pinMode(13, OUTPUT);
}

void loop()
{
    long total = key1.capacitiveSensor(30);
    button = 0;
    if (total > range) button=1;
    if (button==1) digitalWrite(13,
HIGH);
    else digitalWrite(13, LOW);
}
```

Завантажити програму в мікроконтролер і перевірити реакцію на дотик до кнопки. Поекспериментувати з чутливістю спрацювання кнопки, підібравши значення, при якому кнопка реагує на дотик, і при якому реагує на наближення пальця до кнопки.

5. Зробити висновки. Звіт повинен містити: титульний лист; тему, мету роботи; порядок виконання; створені програми; висновки.

Контрольні запитання

1. Яка функція налаштовує режим роботи піна на вхід?
2. Яка функція дозволяє зчитати стан цифрового входу?
3. Яка функція дозволяє змінити стан цифрового виходу?

Лабораторна робота 2. Організація зчитування сигналів з сенсорів

Мета роботи: навчитися програмувати плату Arduino Uno для зчитування сигналу з сенсорів.

Теоретичні відомості

Мікроконтролер в Arduino Uno містить 3 таймери. Вони можуть використовуватись для отримання імпульсного сигналу від датчиків (сенсорів).

Мікроконтролер ATmega328P містить два 8-розрядні таймери та один 16-розрядний. Таймер 1 має реєстр захоплення OCR1, куди записується вміст таймера TCNT1 по події захоплення - зміні сигналу на вході ICP1 або виході аналогового компаратора.

Режим захоплення (capture). У режимі захвату 16-ти бітне значення таймера (TCNT1) захоплюється в реєстр OCR1 при кожній події на вході ICP1 або виході аналогового компаратора. Подія для захоплення задається в реєстрі TCCR1B (біт ICES1) та ACSR (біт ACIC). Для спрощення роботи можна використовувати бібліотеку Timer1.

Режим захоплення використовується для вимірювання тривалості між двома подіями, наприклад періоду, тривалості імпульсу, скважності і т.д.

Приклад 1. Вимірювання періоду дискретного сигналу (рис. 2.1)

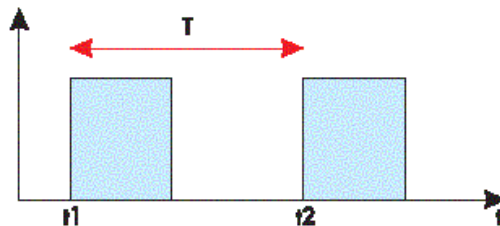


Рис. 2.1. Вимірювання періоду

Приклад 2. Вимірювання періоду з усередненням результату (рис. 2.2)

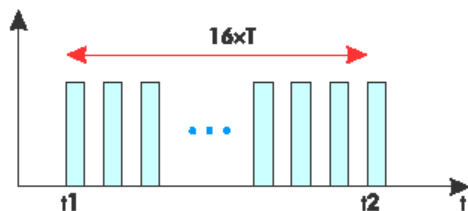


Рис. 2.2. Вимірювання періоду з усередненням

Приклад 3. Вимірювання тривалості імпульсу (рис.2.3)

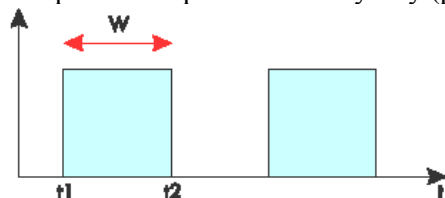


Рис. 2.3. Вимірювання тривалості імпульсу

Приклад 4. Вимірювання скважності імпульсів (рис. 2.4)

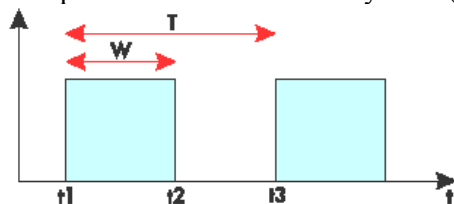


Рис. 2.4. Вимірювання скважності імпульсів

Аналогово-цифровий перетворювач (АЦП, англ. Analog-to-digital converter, ADC) - пристрій, що перетворює вхідний аналоговий сигнал в цифровий код (цифровий сигнал).

У мікроконтролері ATmega328P є 10-розрядний АЦП. Вхідна напруга конвертується в 10-розрядне двійкове значення. Мінімальне значення відповідає 0, а максимальне - опорній напрузі V_{ref} . Отриманий результат перетворення повертається функцією `analogRead(pin)` як число в діапазоні 0...1023. Результат перетворення можна розрахувати за формулою

$$ADC = \frac{V_{in} \cdot 1023}{V_{ref}}$$

Внаслідок перехідних процесів швидкість АЦП є обмеженою й для отримання 10-бітних результатів перетворення максимальна частота – 15 тисяч вибірок за секунду.

План роботи

1. Навчитися підключати датачі з дискретними вихідними сигналами до Arduino та обробляти дані цих датачів.
2. Навчитися підключати датачі з аналоговими вихідними сигналами до Arduino та обробляти дані цих датачів.

Порядок виконання роботи

1. Підключити до плати Arduino ультразвуковий датчик вимірювання відстані HC-SR04. Вивід Echo датчика підключити до виводу 12 плати Arduino; вивід Trig – до виводу 11; вивід Vcc – до виводу 5 V; вивід GND – до одного з виводів GND на платі Arduino.

Завантажити в МК наступну програму:

```
#define echoPin 12 // Echo Pin
#define trigPin 11 // Trigger Pin
#define LEDPin 13 // Onboard LED
char rec;
int maximumRange = 300; // максимальна відстань
int minimumRange = 0; // мінімальна відстань
float duration, distance; // тривалість, потрібна для розрахунку відстані

void setup() {
  Serial.begin (9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(LEDPin, OUTPUT); // Use LED indicator (if required)
}
```

```

void loop() {
    /* The following trigPin/echoPin cycle is
    used to determine the distance of the nearest
    object by bouncing soundwaves off of it. */
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);

    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);

    digitalWrite(trigPin, LOW); //надіслали
імпульс
    duration    = pulseIn(echoPin, HIGH);
    //чекаємо відлуння й записуємо час у змінну
    distance = duration/58.2;
    if (distance >= maximumRange || distance
    <= minimumRange){ //якщо значення помилкове
    (поза діапазоном вимірювання)
        Serial.println("error");
        Serial.print("\n\r");
        digitalWrite(LEDPin, HIGH);
        delay(50);
    }
    else {
        Serial.println(distance);
        Serial.print("\n\r");
        digitalWrite(LEDPin, LOW);
        delay(250);
    }
    delay(150);

byte l[255];
byte count=0;
int i;
if (Serial.available()>0){
while(Serial.available()>0){
    digitalWrite(LEDPin, HIGH);
    l[count] = Serial.read();

```

```

    count++;}
for(i=0;i<count;i++){
    Serial.write(l[i]);};
    Serial.println();
    Serial.print("\n\r");
    delay(500);}
}

```

Відкрити монітор порту. Піднести руку навпроти давача на відстані 20 – 50 см. Спостерігати за виміряними значеннями відстані, що виводиться на екран.

2. Підключити підстроювальний або змінний резистор за схемою подільника напруги (потенціометра) на рис. 2.5.

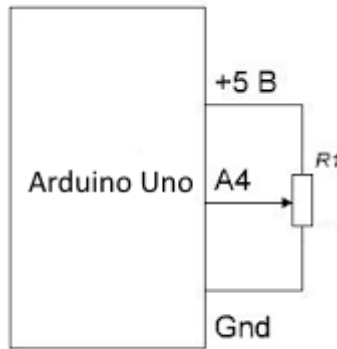


Рис. 2.5. Схема підключення змінного резистора до Arduino Uno

Відповідно до напруги на ковзному контакті (0...5 В) передавати у послідовний інтерфейс значення змінної `norm`, яка зберігає число від 0 до 100, яке прямопропорційно залежить від напруги.

```

    int sensorPin = A4;    // аналоговий вхід
    int sensorValue = 0;  // значення сигналу
від давача
    int norm=0; // нормоване значення сигналу
з давача
void setup() {
    Serial.begin(9600);    // налаштовуємо

```


послідовний порт для зв'язку з ПК

```
}  
void loop() {  
    sensorValue = analogRead(sensorPin);  
    // зчитуємо значення з давача  
    norm=map(sensorValue, 0, 1023, 0, 100);  
    // нормуємо сигнал в межах від 0 до 100  
    Serial.println(norm);           // виводимо  
    нормоване значення сигналу в порт  
    delay(500); // затримка 0,5 с  
}
```

3. Замість змінного резистора підключити до аналогового входу подільник, утворений постійним резистором й давачем освітленості (фоторезистором) згідно рис. 2.6. Змінюючи освітленість фоторезистора, спостерігати за значенням змінної *norm*.

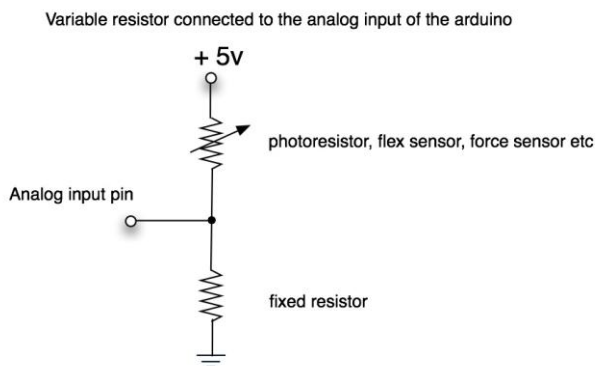


Рис. 2.6. Схема підключення фоторезистора до Arduino Uno

4. Оформити звіт про виконання лабораторної роботи. Звіт повинен містити: назву та мету лабораторної роботи; тексти програм з коментарями; висновок про виконання роботи.

Контрольні запитання

1. Скільки таймерів має мікроконтролер ATmega328P?
2. Яка розрядність модуля АЦП мікроконтролера ATmega328P?
3. Яка кількість каналів модуля АЦП в Arduino Uno?

4. Якою функцією можна виміряти тривалість імпульсу?
5. Якою функцією можна отримати результат АЦП?

Лабораторна робота 3. Реалізація програмного керування двигуном постійного струму

Мета роботи: ознайомитися зі способами керування швидкістю обертів двигунів постійного струму. Навчитися створювати програми керування швидкістю обертів ДПС для мікроконтролера.

Теоретичні відомості

У мехатронних системах одним з найпоширеніших виконавчих пристроїв є двигун постійного струму (ДПС). Конструкції ДПС відрізняються великою різноманітністю. У цій лабораторній роботі розглядається колекторний ДПС, будову якого схематично показано на рис. 3.1.

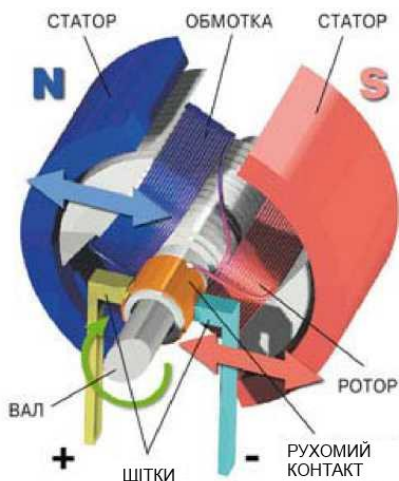


Рис. 3.1. Будова колекторного ДПС з двополюсним статором і двополюсним ротором

Найпростіший колекторний ДПС з двополюсним статором і з двополюсним ротором, складається з одного постійного магніту на статорі, з одного електромагніту з явно вираженими полюсами на роторі (якорі), щітково-колекторного вузла з двома пластинами (ламельями) і двома щітками.

Широтно-імпульсна модуляція

Регулювати швидкість обертів ДПС при зміні навантаження на валу можна зміною напруги, що подається на коло якоря. У сучасних мікропроцесорних САР напругу змінюють не безперервно, а дискретно за допомогою широтно-імпульсної модуляції (ШІМ). Принцип ШІМ пояснюється на рис. 3.2. При ШІМ вихідний сигнал являє собою імпульсний сигнал постійної частоти і змінної скважності. Скважність – це відношення періоду проходження імпульсів T до їх тривалості τ . Обернена величина – коефіцієнт заповнення:

$$s = \frac{\tau}{T}.$$

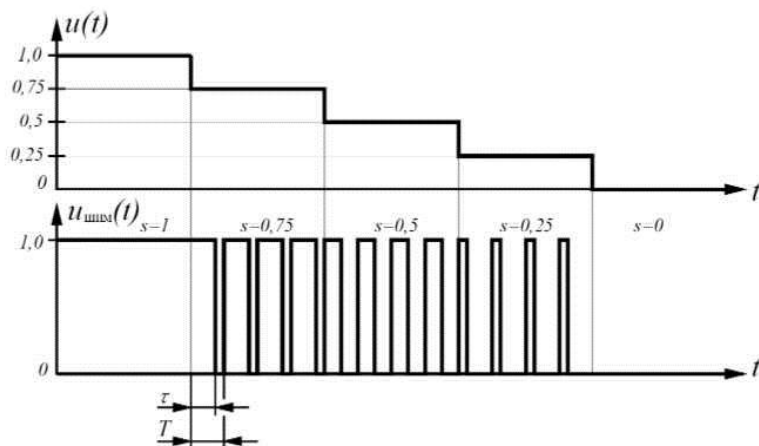


Рис. 3.2. Широтно-імпульсна модуляція

Широтно-імпульсна модуляція (ШІМ - англ. *pulse-width modulation, PWM*), або модуляція за тривалістю імпульсів (англ. *pulse-duration modulation, PDM*) – процес керування шириною (тривалістю) високочастотних імпульсів по закону, який задає низькочастотний сигнал. В електроніці це може бути керування середнім значенням вихідної напруги шляхом зміни тривалості замкнутого стану електронного (електромеханічного) ключа.

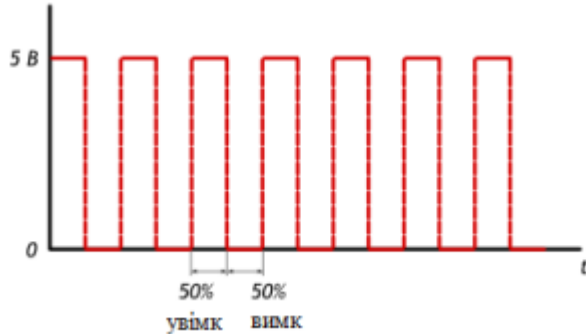
Аналогова ШІМ. ШІМ-сигнал генерується аналоговим

компаратором, на один вхід якого подається опорний сигнал значно більшої частоти, ніж модулюючий у вигляді «трикутника» або «пилки», а на іншій – модулюючий неперервний аналоговий сигнал. Частота вихідних імпульсів ШІМ відповідає частоті «зубів» пилки. В ту частину періоду, коли сигнал на позитивному вході вище сигналу на негативному вході, на виході виходить одиниця, в іншу, коли сигнал на позитивному вході нижче сигналу на негативному вході - нуль.

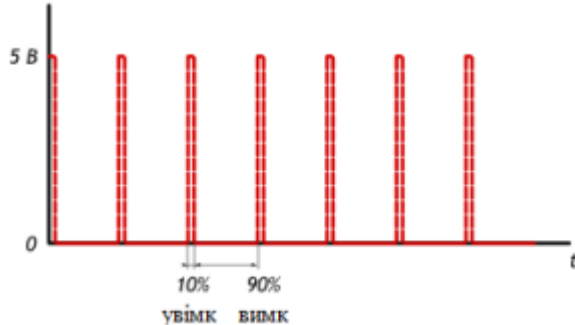
Цифрова ШІМ. У двійковій цифровій техніці, виходи в якій можуть приймати тільки одне з двох значень, наближення бажаного середнього рівня вихідного сигналу за допомогою ШІМ є абсолютно природним. Схема така ж проста: пилкоподібний сигнал генерується N -бітовим лічильником.

Розглянемо кілька прикладів при напрузі живлення $V_{cc}=5\text{ В}$.

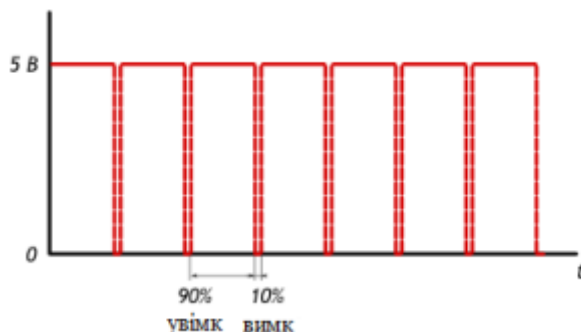
50% — еквівалент 2,5 В



10% — еквівалент 0,5 В



90% — еквівалент 4,5 В



Для подачі ШІМ сигналу певної скважності на дискретний вихід Arduino використовується функція `analogWrite()`.

`analogWrite()` – формує задану "аналогову" напругу на виводі у вигляді ШІМ-сигналу. Після виклику `analogWrite()`, на виводі буде безперервно генеруватися ШІМ-сигнал із заданим коефіцієнтом заповнення до наступного виклику функції `analogWrite()` (або до моменту виклику `digitalRead()` або `digitalWrite()`, що взаємодіють з цим же виводом). Частота ШІМ становить приблизно **490 Гц**. На платі **Uno** та подібних виводи 5 і 6 мають частоту ШІМ приблизно **980 Гц**.

На більшості плат Arduino (на базі мікроконтролерів ATmega168 або ATmega328) функція `analogWrite()` працює з виводами 3, 5, 6, 9, 10 і 11. На **Arduino Mega** функція працює з виводами з 2 по 13 і 44 – 46.

Функція `analogWrite()` не має нічого спільного з аналоговими входами і функцією `analogRead()`.

Синтаксис:

```
analogWrite(pin, value)
```

Параметри:

`pin` - номер ШІМ-виходу

`value` - значення скважності від 0 (завжди "0") до 255

(завжди "1").

Найпростіша схема керування двигуном постійного струму складається з польового транзистора, на затвор якого подається ШІМ сигнал. Транзистор в даній схемі виконує роль

електронного ключа, який комутує один з виводів двигуна на землю. Транзистор відкривається на час тривалості імпульсу.

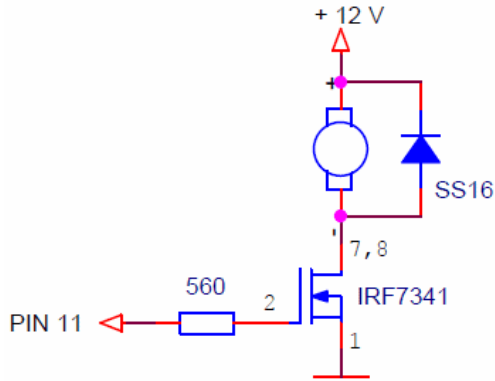


Рис. 3.3. Схема керування швидкістю обертів ДПС з використанням польового транзистора

План роботи

1. Ознайомитися зі способами керування швидкістю обертів ДПС.
2. Скласти програму для керування швидкістю обертів ДПС.

Порядок виконання роботи

1. Ознайомитися з теоретичними відомостями.
2. Завантажити середовище Arduino IDE.
3. Скласти на макетній платі наступну схему рис. 3.3.
4. В середовищі Arduino IDE створити нову програму.

```
int led = 11; // ШІМ-вивід, до якого
підключений затвор транзистора
int speed = 0; // ця змінна відповідає за
швидкість обертання
int fadeAmount = 5; // Крок зміни
швидкості
void setup ()
{ // Налаштовуємо вивід 9 як дискретний
вихід
pinMode (led, OUTPUT);
```

```

    }
    void loop ()
    { // Встановлюємо швидкість обертання
      двигуна
      analogWrite(led, speed); // Збільшуємо
      поточне значення швидкості обертання
      speed = speed + fadeAmount; // Коли
      швидкість стає максимальною / мінімальною -
      починаємо її знижувати / підвищувати
      if (speed == 0 || speed == 255)
      {
        fadeAmount = -fadeAmount;
      }
      // Пауза 30 мілісекунд
      delay (30);
    }

```

5. Скомпілювати програму та переконатися у відсутності помилок.

6. Підключити мікроконтролерну плату до USB-порта комп'ютера.

7. У середовищі Arduino IDE вибрати тип плати та порт, до якого вона підключена. Завантажити створену програму в мікроконтролер. Переконатися в коректності роботи програми.

8. Самостійно створити програму для задання постійної швидкості обертання ДПС на рівні 95% від верхньої межі діапазону регулювання. Завантажте програму в МК та переконайтеся в правильності її роботи.

9. Оформити звіт про виконання лабораторної роботи. Звіт повинен містити: назву та мету лабораторної роботи; тексти програм з коментарями; висновок про виконання роботи.

Контрольні запитання

1. Яка будова найпростішого колекторного ДПС?
2. Які способи керування швидкістю ДПС Вам відомі?
3. Що таке широтно-імпульсна модуляція?
4. Що таке скважність імпульсів?
5. Як можна змінювати швидкість обертання ДПС за допомогою ШІМ?

Лабораторна робота 4. Дослідження роботи сервоприводів та реалізація циклограми

Мета роботи: ознайомитися зі способами керування положенням сервоприводів. Навчитися створювати програми керування швидкістю повороту і положенням вихідного вала сервопривода.

Теоретичні відомості

Під сервоприводом найчастіше розуміють механізм з електродвигуном, який можна попросити повернутися в заданий кут і утримувати це положення. Однак, це не зовсім повне визначення. Якщо сказати повніше, сервопривід - це привід з управлінням через негативний зворотний зв'язок, що дозволяє точно керувати параметрами руху. Сервоприводом є будь-який тип механічного приводу, що має в складі давач (положення, швидкості, зусилля тощо) і блок керування приводом, який автоматично підтримує необхідні параметри згідно заданому завданню. Тобто сервопривод отримує на вхід значення керуючого параметра, наприклад, кут повороту. Блок управління порівнює це значення зі значенням на своєму датчику. На основі результату порівняння привод виконує певну дію, наприклад: поворот, прискорення або сповільнення так, щоб значення з внутрішнього давача стало якомога ближче до значення зовнішнього керуючого параметра.

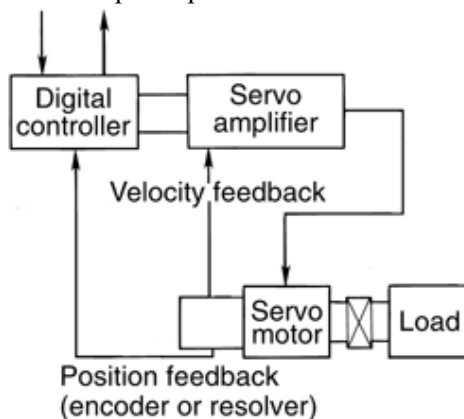


Рис. 4.1. Структурна схема сервопривода з давачем положення та швидкості

Найбільш поширені сервоприводи, які утримують заданий кут і сервоприводи, що підтримують задану швидкість обертання.



Рис. 4.2. Сервоприводи

Отже, сервопривод – це регульований редукторний електродвигун. Він зазвичай складається з приводного механізму з двигуном постійного струму, плати управління і потенціометра, котрий забезпечує зворотний зв'язок.

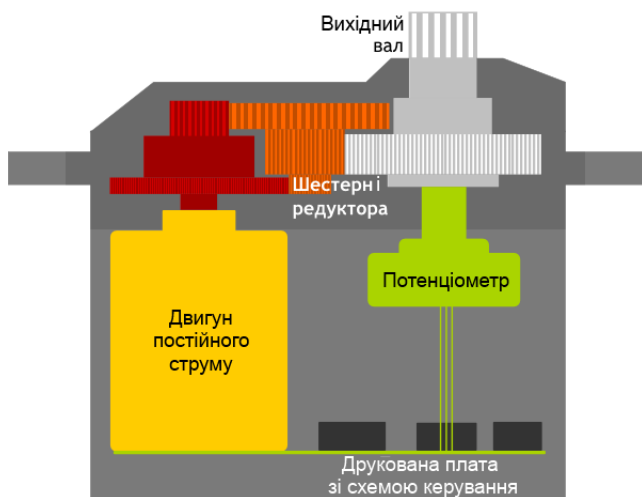


Рис. 4.3. Будова сервопривода

Управління сервоприводом. Інтерфейс керуючих сигналів

Керуючий сигнал для сервопривода – імпульси постійної частоти і змінної ширини. Те, яке положення повинен зайняти сервопривод, залежить від довжини імпульсів. Коли сигнал надходить в керуючу схему, наявний у ній генератор імпульсів виробляє свій імпульс, тривалість якого визначається через потенціометр. Інша частина схеми порівнює тривалість двох імпульсів. Якщо тривалість різна, включається електродвигун. Напрямок обертання визначається тим, який з імпульсів коротший. Якщо довжини імпульсів рівні, електродвигун зупиняється. Найчастіше в аматорських сервоприводів імпульси виробляються з частотою 50 Гц. Це означає, що імпульс випускається і приймається раз в 20 мс. Зазвичай при цьому тривалість імпульсу 1520 мкс означає, що сервопривод повинен зайняти середнє положення. Збільшення або зменшення довжини імпульсу змусить сервопривод повернутися за годинниковою або проти годинникової стрілки відповідно. При цьому існують верхня і нижня межі тривалості імпульсу.

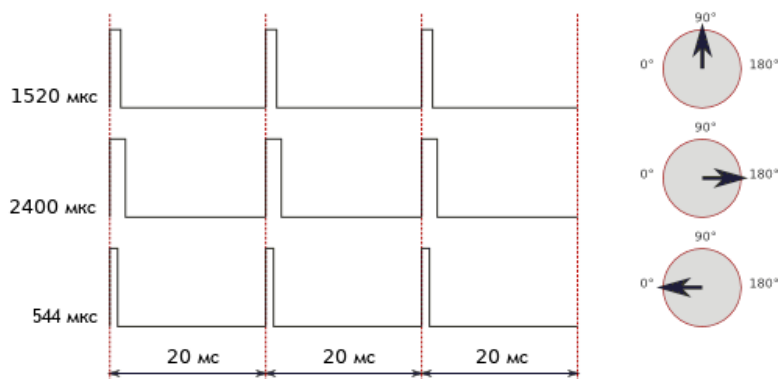


Рис. 4.4. Залежність кута повороту сервопривода від тривалості керуючих імпульсів

У бібліотеці Servo для Arduino за замовчуванням виставлені наступні значення довжин імпульсу: 544 мкс – для 0° і 2400 мкс – для 180°. На конкретному пристрої заводські налаштування можуть відрізнятися від стандартних. Також варто відзначити, що це всього лише загальноприйняті довжини.

Навіть у рамках однієї і тієї ж моделі сервоприводу може існувати похибка, що допускається при виробництві, яка призводить до того, що робочий діапазон довжин імпульсів трохи відрізняється. Для точної роботи кожен конкретний сервопривод повинен бути відкалібрований: шляхом експериментів необхідно підібрати коректний діапазон, характерний саме для нього. При формуванні сигналу керування для сервопривода важливою є довжина імпульсів, а не частота їх появи. 50 Гц – це норма, але сервопривод буде працювати коректно і при 40, і при 60 Гц. При цьому слід мати на увазі, що при сильному зменшенні частоти він може працювати ривками і на зниженій потужності, а при сильному завищенні частоти може перегрітися і вийти з ладу.

Сервоприводи малої потужності (наприклад, SG90) можна безпосередньо підключати до плати Arduino, для підключення більш потужних – слід використовувати зовнішнє джерело живлення.

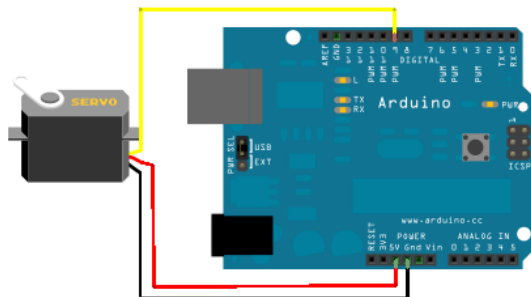


Рис. 4.5. Підключення сервопривода малої потужності до мікроконтролерної плати Arduino Uno

Бібліотека Servo дозволяє здійснювати програмне керування сервоприводами. Управління здійснюється наступними функціями:

`attach()` – приєднує об'єкт до конкретного виводу плати. Можливі два варіанти синтаксису для цієї функції: `servo.attach(pin)` і `servo.attach (pin, min, max)`. При цьому `pin` -

номер виводу, до якого приєднують сервопривод, min і max - довжини імпульсів в мікросекундах, що відповідають за кути повороту 0° і 180°. За замовчуванням виставляються рівними 544 мкс і 2400 мкс відповідно.

write() - віддає команду сервоприводу прийняти деяке значення параметра. Синтаксис наступний: servo.write (angle), де angle - кут, на який повинен повернутися сервопривод.

writeMicroseconds() - віддає команду надіслати на сервопривод імпульс певної довжини, є низькорівневим аналогом попередньої команди. Синтаксис наступний: servo.writeMicroseconds(uS), де uS – довжина імпульсу в мікросекундах.

read() - читає поточне значення кута, в якому знаходиться сервопривод. Синтаксис наступний: servo.read(), повертається ціле значення від 0 до 180.

attached() - перевірка, чи була приєднаний об'єкт до конкретного виводу. Синтаксис наступний: servo.attached(), повертається true, якщо змінна була приєднана до якого-небудь виводу, або false в зворотному випадку.

detach() - виконує дію, зворотну дії attach(), тобто від'єднує об'єкт від виводу, з яким він був асоційований. Синтаксис наступний: servo.detach().

Приклад програми з використанням бібліотеки Servo

```
#include <Servo.h>
Servo myservo;
void setup()
{
    myservo.attach(9);
}
void loop()
{
    myservo.write(90); // встановлюємо
сервопривод в середнє положення
    delay(500);
    myservo.write(0); // встановлюємо
сервопривод в крайнє лівє положення
    delay(500);
```

```
myservo.write(180); // встановлюємо
сервопривод в крайнє праве положення
delay(500);
}
```

План роботи

1. Скласти програму для керування положенням сервопривода.
2. Реалізувати циклограму положення вала сервопривода.
3. Програмно обмежити швидкість повороту вала сервопривода.

Порядок виконання роботи

1. Складіть схему за рис. 4.5, використовуючи сервопривод SG90.
2. Завантажте програму з прикладу бібліотеки Servo.
3. Напишіть програму для реалізації наступної циклограми.

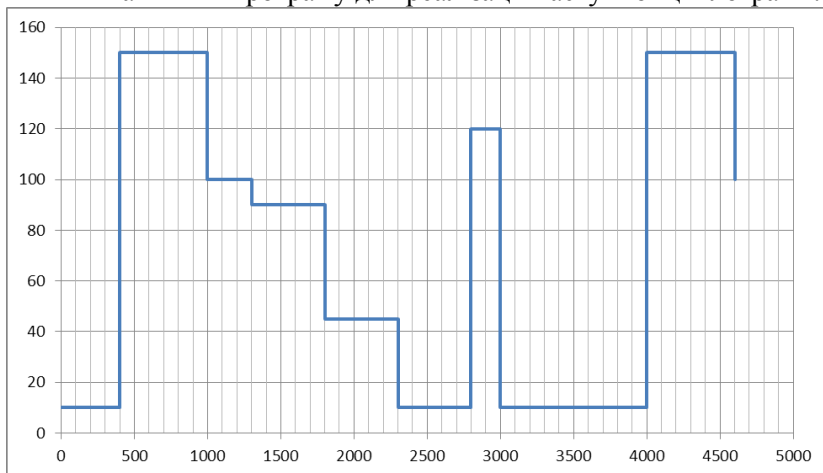


Рис. 4.6. Задана циклограма роботи

Вісь абсцис – час в мілісекундах, вісь ординат – положення вала сервопривода в градусах.

3. Для того, щоб мати змогу змінювати швидкість обертання використайте бібліотеку VarSpeedServo. Завантажити в МК приклад VarSpeedServo-Sweep, змінюючи значення швидкості. Переконайтеся в правильності роботи програми та точності позиціонування сервопривода.

4. Оформити звіт про виконання лабораторної роботи.

Звіт повинен містити: назву та мету лабораторної роботи; тексти програм з коментарями; висновок про виконання роботи.

Контрольні запитання

1. Що таке сервопривод?
2. Назвіть основні елементи сервопривода.
3. Як реалізоване керування положенням вала сервопривода, що використано в цій лабораторній роботі?
4. Яка бібліотека дає змогу обмежувати швидкість повороту сервопривода?

Лабораторна робота 5. Дослідження роботи маніпулятора з дистанційним управлінням

Мета роботи: Ознайомитися з принципом управління роботом-маніпулятором з 6-ма ступенями свободи. Навчитися створювати програми керування для дистанційного управління маніпулятором.

Теоретичні відомості

У процесі автоматизації промислових підприємств важливим є використання роботизованих комплексів, що складаються з механічних маніпуляторів та систем управління ними. Застосування промислових роботів-маніпуляторів дозволяє виключити вплив людського фактора на виробництві, підвищити точність виконання технологічних операцій, певною мірою зменшити вплив шкідливих факторів на персонал, скоротити площу виробничих приміщень і забезпечити безперебійну роботу виробництва.

Промисловий робот – автоматична машина, що складається з маніпулятора і пристрою програмного керування його рухом, призначений для заміни людини при виконанні основних і допоміжних операцій у виробничих процесах.

Маніпулятор – сукупність просторового важільного механізму і системи приводів, що здійснює під керуванням програмованого автоматичного пристрою чи людини-оператора дії (маніпуляції), аналогічні діям руки людини.

Промислові роботи призначені для заміни людини при виконанні основних і допоміжних технологічних операцій у процесі промислового виробництва. Гнучкі автоматизовані

виробництва, які створюються на базі промислових роботів, дозволяють вирішувати задачі автоматизації на підприємствах із широкою номенклатурою продукції при дрібносерійному і штучному виробництві.

Необхідність дослідження та вдосконалення систем управління маніпуляційними роботами насамперед зумовлена їх широким застосуванням. Подібні пристрої використовуються в будівельній галузі (крани-маніпулятори), металургії (прокатні стани, кувальні маніпулятори), гірничодобувній промисловості (бурильні машини), хімічній промисловості (маніпулятори для роботи з токсичними і радіоактивними матеріалами), суднобудівній та авіаційній галузях (зварювальні та складальні маніпулятори, маніпулятори для різання металів) тощо.

Маніпулятор промислового робота повинен забезпечувати рух вихідної ланки і, закріпленого в ній, об'єкта маніпулювання в просторі за заданою траєкторією і з заданою орієнтацією. Промисловий робот із шістьма ступенями свободи є складною автоматичною системою. У реальних конструкціях промислових роботів часто використовуються також механізми зі ступенями свободи менше шести. Найбільш прості маніпулятори мають три, рідше два, рухи. Такі маніпулятори значно дешевші у виготовленні та експлуатації, але вимагають специфічних вимог до організації робочого простору.

Розглянемо для прикладу структурну і функціональну схеми промислового робота з трьома рухомими ланками. Основний механізм руки маніпулятора складається з нерухомої ланки 0 і трьох рухомих ланок 1, 2 і 3 (рис. 5.1). Механізм цього маніпулятора відповідає циліндричній системі координат. У цій системі ланка 1 може обертатися відносно ланки 0 (відносне кутове переміщення φ_{10}), ланка 2 переміщається по вертикалі відносно ланки 1 (відносне лінійне переміщення S_{21}) і ланка 3 переміщається в горизонтальній площині відносно ланки 2 (відносне лінійне переміщення S_{32}). На кінці ланки 3 закріплений захоплюючий пристрій (захват), призначений для захоплення й утримання об'єкта маніпулювання. Ланки основного важільного механізму маніпулятора утворюють між собою три однорухливі кінематичні пари (одну обертальну A і дві поступальні B та C) і можуть забезпечити переміщення

об'єкта в просторі без керування його орієнтацією.

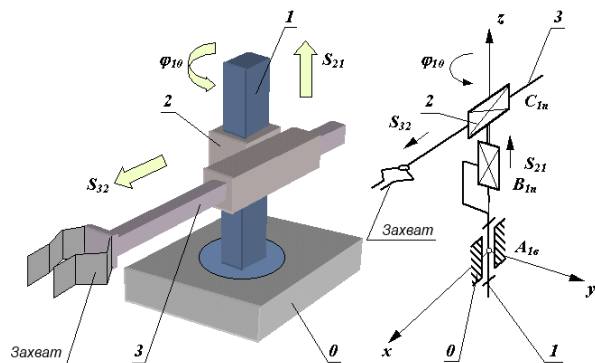


Рис. 5.1. Структурна і функціональна схеми промислового маніпулятора

Для виконання кожного з трьох відносних рухів маніпулятор повинен бути оснащений приводами, що складаються з двигунів, редуктора і системи давачів зворотного зв'язку. Оскільки рух об'єкта здійснюється за заданим законом руху, то в системі повинні бути пристрої, що зберігають і задають програму руху. Перетворення заданої програми руху в сигнали керування приводами u_i здійснюється системою керування. При необхідності вона коректує ці впливи за сигналами Δx_i , що надходять до неї з давачів зворотного зв'язку (рис. 5.2).

Отже, маніпуляційний робот складається з декількох ступенів рухливості (ланок) і приводів, що призводять ланки в рух. У якості приводів робота найчастіше використовуються крокові двигуни або сервоприводи різної потужності. Кроковим двигунам надається перевага, якщо швидкість переміщення ланок робота не є критичним параметром. Наприклад, такий тип приводів може використовуватися при побудові вантажних маніпуляторів. Якщо ж потрібно забезпечити високу швидкість руху робота, то найбільш доцільно використовувати сервоприводи.

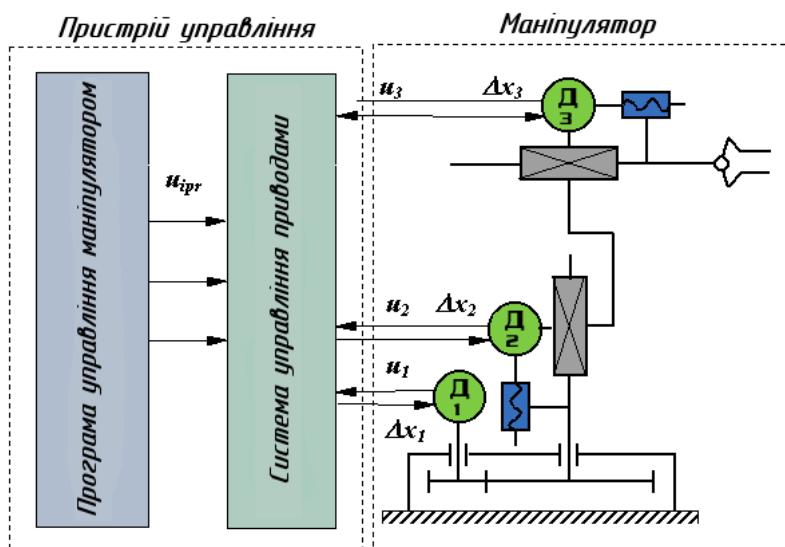


Рис. 5.2. Функціональна схема промислового робота

Процес розробки робота-маніпулятора складається з двох етапів:

- розробка механічної частини робота (вибір матеріалу для виготовлення складових частин, а також типу виконавчих механізмів);
- розробка системи управління маніпулятором (вибір контролера, вибір засобів програмування, розробка алгоритмів керування).

Швидкодію промислових роботів визначають за максимальною швидкістю лінійних переміщень центра захвату маніпулятора. Розрізняють промислові роботи з малою ($V_M < 0,5$ м/с), середньою ($0,5 < V_M < 1,0$ м/с) і високою ($V_M > 1,0$ м/с) швидкодією. Сучасні промислові роботи мають в основному середню і високу швидкодію.

Точність маніпулятора промислового робота характеризується абсолютною лінійною похибкою позиціонування центра захвату. Промислові роботи поділяються на групи з малою ($\Delta r_M < 1$ мм), середньою ($0,1 \text{ мм} < \Delta r_M < 1$ мм) і високою ($\Delta r_M < 0,1$ мм) точністю позиціонування.

Опис лабораторної установки

Досліджуваний маніпулятор, зображений на рис. 5.3, має 6 ступенів свободи. Для приведення в рух ланок маніпулятора в кожному із суглобів встановлено сервопривод MG996R. Перший ступінь рухомості забезпечує основа маніпулятора, другий – плече, третій – лікоть, четвертий – обертання кисті, п'ятий – поворот кисті, шостий – захват.

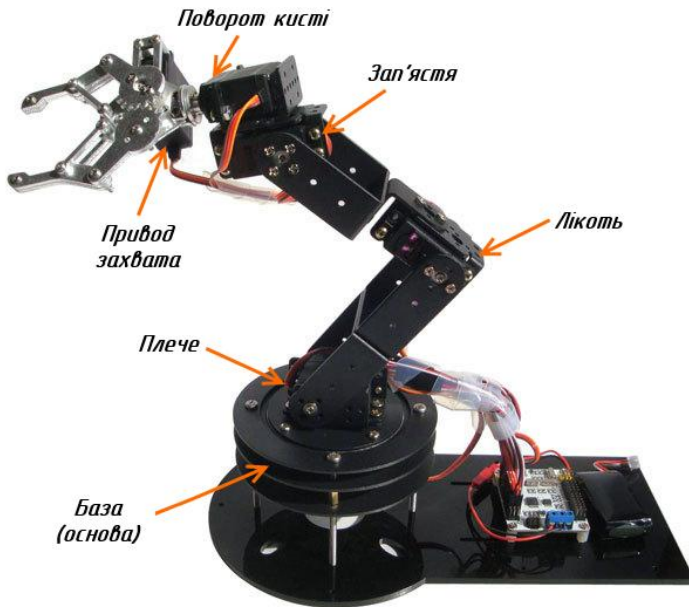


Рис. 5.3. Зовнішній вигляд досліджуваного маніпулятора

Параметри сервоприводів MG996R подані нижче.



Рис. 5.4. Зовнішній вигляд та розміри сервопривода MG996R

Основні характеристики:

З'єднувальний провід: довжина 300мм.

Розміри: 40.7 мм x 19,7 мм x 42.9 мм.

Маса: 55 г

Робоча швидкість: 0.17с / 60 градусів (4.8В без навант.)

Робоча швидкість: 0.14с/ 60 градусів (6.0В без навант.)

Обертальний момент: 9.4 кгс·см (4.8В), 11 кгс·см (6 В)

Джерело живлення: через зовнішній адаптер.

Робоча напруга: 4.8 – 7.2V

Редуктор : металеві шестерні.

Робочий струм при русі: 500 – 900 мА (6 В).

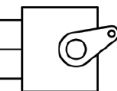
Струм утримання (момент навантаження дорівнює крутному моменту двигуна): 2.5 А (6 В).

Підключення:

PWM=Orange (⏏)

Vcc = Red (+)

Ground=Brown (-)



Для живлення сервоприводів у лабораторній роботі використовується блок живлення 5В, 10А.

Система управління роботом-маніпулятором реалізована на основі мікроконтролерної плати Arduino Mega 2560.

План роботи

1. Ознайомитися з будовою та принципами управління роботами-маніпуляторами.
2. Ознайомитися з будовою лабораторної установки.
3. Визначити межі безпечних кутів повороту (робочу зону) маніпулятора.

Порядок виконання роботи

1. Ознайомитися з теоретичними відомостями.
2. Завантажити середовище Arduino IDE.
3. Створити нову програму.

```
#include <VarSpeedServo.h>
VarSpeedServo myservo1, myservo2, myservo3,
myservo4, myservo5, myservo6;    // create servo
object to control a servo
byte a1,a2,a3,a4,a5,a6; //angle of Servo
byte oa1,oa2,oa3,oa4,oa5,oa6; //Old angle of
Servo

boolean serialflag = 0;

void setup()
{
    // initialize serial:
    Serial.begin(9600);
    myservo1.attach(2,900,2365);
    myservo2.attach(3,720,1000);
    myservo3.attach(4,720,2365);
    myservo4.attach(5,720,2300);
    myservo5.attach(10,670,2300);
    myservo6.attach(11,720,2300);
    oa1 = myservo1.read();
    oa2 = myservo2.read();
    oa3 = myservo3.read();
    oa4 = myservo4.read();
    oa5 = myservo5.read();
    oa6 = myservo6.read();
}

void loop()
{
    if (a1 != oa1){setPosServo(myservo1,a1);oa1
= a1;printserial(); }
```

```

        if (a2 != oa2){setPosServo(my servo2,a2);oa2
= a2;printserial(); }
        if (a3 != oa3){setPosServo(my servo3,a3);oa3
= a3;printserial(); }
        if (a4 != oa4){setPosServo(my servo4,a4);oa4
= a4;printserial(); }
        if (a5 != oa5){setPosServo(my servo5,a5);oa5
= a5;printserial(); }
        if (a6 != oa6){setPosServo(my servo6,a6);oa6
= a6;printserial(); }
        if (serialflag == 1) {printserial();
serialflag = 0;}
        //printserial(); delay(200);
    }

    void setPosServo(VarSpeedServo myservo,int
angle){
        int i = 1;
        int oldangle = myservo.read();
        if (oldangle > angle) {i=-1;}
        for (int a = oldangle; a!= angle; a += i ){
            myservo.slowmove(a,15);        // tell servo
to go to position in variable 'pos'
            delay(15);
        }
    }
    void serialEvent() {
        byte myservo;
        while (Serial.available()) {
            // get the new byte:
            byte inByte = (byte)Serial.read();
            if ((inByte >= 201) && (inByte<=206) ) {
                myservo = inByte - 200;
            }
            if (inByte <= 180) {
                if (myservo == 1) {a1 = inByte;}
                if (myservo == 2) {a2 = inByte;}
                if (myservo == 3) {a3 = inByte;}
                if (myservo == 4) {a4 = inByte;}
                if (myservo == 5) {a5 = inByte;}
                if (myservo == 6) {a6 = inByte;}
                serialflag = 1;
            }
        }
    }

```

```

        myservo = 0;
    }
}

void printserial() {
    delay(20);
    Serial.print(myservo1.read()); // Movement
Data
    Serial.print(",");
    Serial.print(myservo2.read()); // Movement
Data
    Serial.print(",");
    Serial.print(myservo3.read()); // Movement
Data
    Serial.print(",");
    Serial.print(myservo4.read()); // Movement
Data
    Serial.print(",");
    Serial.print(myservo5.read()); // Movement
Data
    Serial.print(",");
    Serial.print(myservo6.read()); // Movement
Data
    Serial.println();
    delay(20);
}

```

4. Скомпілювати програму та переконатися у відсутності помилок.

5. Підключити мікроконтролерну плату до USB-порту комп'ютера.

6. У середовищі Arduino IDE вибрати тип плати та порт, до якого вона підключена. Завантажити створену програму в мікроконтролер.

7. Запустити RoboArm Control 2.

8. З дозволу викладача ввімкнути живлення стенда.

9. Плавню переміщуючи повзунки смуг прокрутки, спостерігати за переміщенням маніпулятора.

10. Встановити межі кута повороту кожного сервопривода. Результати записати в табл. 5.1.

Таблиця 5.1

№ з/п	Сервопривод	Нижня межа кута повороту	Верхня межа кута повороту
1	База		
2	Плече		
3	Лікоть		
4	Зап'ястя		
5	Поворот кисті		
6	Захват		

11. Вимкніть живлення стенда.

12. Від'єднайте плату Arduino від комп'ютера.

13. Зробити висновки. Звіт повинен містити: титульний лист; тему, мету роботи; порядок виконання; створені програми; заповнену таблицю 5.1; висновки.

Контрольні запитання

1. Що таке промисловий робот?

2. Що таке маніпулятор?

3. Яка сфера застосування промислових роботів і маніпуляторів?

4. Наведіть приклад функціонально схеми промислового робота?

5. Які типи приводів використовують у промислових маніпуляторах?

6. Як класифікують промислові роботи за точністю позиціонування?

7. Що таке ступені свободи маніпулятора?

8. Які приводи маніпулятора застосовують в лабораторному стенді? Який принцип управління цими приводами?

Лабораторна робота 6. Реалізація блокування роботи маніпулятора при виявленні перешкод

Мета роботи: навчитися програмувати зупинку роботи мехатронної системи у процесі виявлення в робочій зоні тіла людини або іншої перешкоди за допомогою датчиків руху. Навчитися налаштовувати чутливість датчиків руху.

Теоретичні відомості

Для виявлення в робочій зоні маніпулятора сторонніх об'єктів широко застосовують датчики руху, датчики відстані, датчики присутності об'єкта, відеокамери в поєднанні з засобами машинного зору.

Датчик руху фіксує переміщення об'єктів і використовується для контролю або автоматичного запуску необхідних дій у відповідь на переміщення об'єкта. На нерухомі об'єкти датчики руху, як правило, не реагують. Найчастіше використовуються інфрачервоні, ультразвукові, радіохвильові датчики руху.

Для виявлення людського тіла здебільшого використовують пасивні інфрачервоні давачі руху. Вони фіксують зміну інфрачервоного випромінювання, що потрапляє в датчик з різних точок простору (зміну теплового фону), й здатні виявити рух теплокровних тварин або інших об'єктів, які досить інтенсивно випромінюють у діапазоні хвиль, відмінному від оточення.

Давач руху HC-SR501

Для виконання лабораторної роботи використовується пасивний інфрачервоний давач руху HC-SR501 (рис. 6.1.).

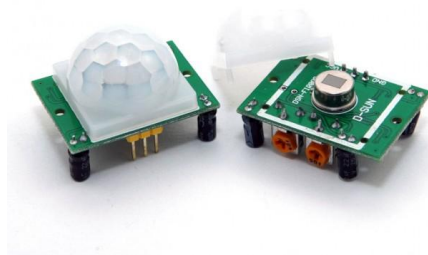


Рис. 6.1. Зовнішній вигляд давача руху HC-SR501

Він складається з піроелектричного датчика (містить у собі 2 піроелектричні елементи), лінзи Френеля та схеми вторинного перетворення сигналу. Лінза Френеля забезпечує *значну* зміну освітленості різних частин піроелектричного елемента при *незначному* русі теплового об'єкта внаслідок *дифракції* й формування максимумів та мінімумів. Різниця

напруг з двох елементів опрацьовується мікросхемою BISS0001, в результаті на виході датчика при виявленні руху зміниться логічний рівень сигналу.



Рис. 6.2. Призначення контактів датчика руху HC-SR501
Активний рівень сигналу на виході утримується впродовж часу, що задається підстроювальним резистором (рис. 6.2.). Інший підстроювальний резистор задає чутливість давача.

План роботи

1. Ознайомитися з документацією на давач руху HC-SR501.
2. Написати програму, що реалізує зупинку лабораторного маніпулятора при виявленні руху руки людини в зоні дії маніпулятора.

Порядок виконання роботи

1. Відкрити документацію на датчик руху HC-SR501 та знайти межі тривалості імпульсу, що генерується при виявленні руху. За допомогою викрутки задати мінімальну тривалість.
2. Скласти схему та написати програму для Arduino, яка запалює світлодіод при виявленні датчиком руху та відлагодити її роботу.
3. У програму в попередній роботі додати перевірку сигналу з датчика руху та зупинку переміщення маніпулятора у випадку виявлення руху.
4. Після перевірки програми та правильності підключення датчика викладачем завантажити програму в плату керування маніпулятором та перевірити правильність роботи.

5. Розробити програму для автоматичного переміщення захвату маніпулятора між двома точками з блокуванням від датчика руху. Перевірити правильність її роботи.

6. Зробити висновки. Звіт повинен містити: титульний лист; тему, мету роботи; порядок виконання; створені програми; висновки.

Контрольні запитання

1. Які види давачів руху ви знаєте?
2. Який принцип дії пасивного інфрачервоного датчика руху?
3. Яке фізичне явище використовується в датчику HC-SR501 для виявлення руху?
4. Яке призначення підстроювальних резисторів в HC-SR501?
5. Для чого використовується лінза Френеля в PIR датчиках руху?
6. В якому випадку пасивний інфрачервоний датчик може не виявити частину тіла людини, що рухається в полі зору датчика?

Лабораторна робота 7. Розроблення програмного забезпечення роботизованої ділянки

Мета роботи: ознайомитися з принципами програмно-логічного управління роботизованими ділянками. Написати програму для автоматичного управління маніпулятором.

Теоретичні відомості

У процесі програмування роботизованих ділянок виробництва потрібно враховувати взаємний вплив кожної ланки у ділянці на попередню, наприклад, неможливість здійснення зупинки одного конвеєра без зупинки попереднього конвеєра, з якого в нормальному режимі роботи повинні забиратись об'єкти на конвеєр, який необхідно зупинити.

Приклад автоматизованої виробничої ділянки показано на рис. 8.1. Заготовка, що прибуває у ділянку на вхідному конвеєрі, переміщається у робочу зону верстата. При цьому слід передбачити зупинку конвеєра до того часу, поки маніпулятор не захопить заготовку. Робот виконує спочатку операцію

переміщення заготовки на вхід у верстат. Потім відбувається обробка заготовки у верстаті. Після цього заготовка переміщається на вихід верстата. Потім, якщо робот вільний, він переносить оброблену заготовку на вихідний конвеєр виробничої ділянки.



Рис. 8.1. Схема досліджуваної роботизованої ділянки

План роботи

1. Ознайомитися з будовою роботизованої ділянки в лабораторній установці.
2. Написати та відлагодити програму для керування роботизованою ділянкою.

Порядок виконання роботи

1. Ознайомитися з схемою підключення приводів роботизованої ділянки до плати Arduino Mega та силовими елементами для керування приводами.
2. Написати програму для керування маніпулятором роботизованої ділянки (забір об'єкту з основи станда й перенесення на каретку).
3. У середовищі Arduino IDE вибрати тип плати та порт, до якого вона підключена. Завантажити створену програму в мікроконтролер.
4. З дозволу викладача увімкнути живлення станда та відлагодити роботу програми.
5. Написати та відлагодити програму для керування рухомою кареткою роботизованої ділянки.
6. На підставі розроблених програм створити програму, що керуватиме роботою всієї ділянки: і робота-маніпулятора, і конвеєра або рухомої каретки (об'єкт забирається

маніпулятором, переміщається на каретку, далі вмикається двигун каретки й об'єкт їде до кінцевої точки, каретка зупиняється, цикл повторюється).

7. Перевірити правильність роботи програми та відлагодити її.

8*. Додати в програму функцію блокування роботи всієї механічної складової при виявленні руху датчиком HC-SR501.

9. Оформити звіт про виконання лабораторної роботи. Звіт повинен містити: назву та мету лабораторної роботи; тексти програм з коментарями; висновок про виконання роботи.

Контрольні запитання

1. У чому відмінність розробки програм керування окремими мехатронними елементами та роботизованими ділянками?

2. Якими способами можна виявити надходження об'єкта на початок ділянки?

3. Які способи керування двигном постійного струму ви знаєте?

4. Чим спричинено використання зворотньо включених діодів паралельно якорям двигунів постійного струму під час керування їх швидкістю за допомогою ШІМ сигналу, що надходить на затвор польового транзистора?

Лабораторна робота 8. Використання комп'ютерного зору при керуванні роботом

Мета роботи: навчитися використовувати методи комп'ютерного зору в задачах керування засобами робототехніки.

Теоретичні відомості

OpenCV – одна з найвідоміших бібліотек комп'ютерного зору, що використовується зокрема й в робототехніці. В засобах керування промисловими процесами (в т. ч. виконуваними промисловими роботами) широко використовується протокол Modbus. Дана лабораторна робота передбачає обробку відеопотоку з камери чи файлу на комп'ютері та надсилання команд про зміну положення сервопривода з використанням протоколу Modbus, реалізованому в бібліотеці libmodbus.

Бібліотека OpenCV складається з таких компонентів:

- `opencv_core` - основна функціональність. Включає в себе базові структури, обчислення (математичні функції, генератори випадкових чисел) і лінійну алгебру, DFT, DCT, введення / виведення для XML і YAML і т. д.

- `opencv_imgproc` - обробка зображень (фільтрація, геометричні перетворення, перетворення кольорних просторів і т. д.).

- `opencv_highgui` - простий інтерфейс користувача, введення / виведення зображень і відео.

- `opencv_ml` - моделі машинного навчання (SVM, дерева рішень, навчання зі стимулюванням і т. д.).

- `opencv_features2d` - розпізнавання і опис плоских примітивів (SURF, FAST й інші, включаючи спеціалізований фреймворк).

- `opencv_video` - аналіз руху і відстеження об'єктів (оптичний потік, шаблони руху, усунення фону).

- `opencv_objdetect` - виявлення об'єктів на зображенні (перебування осіб за допомогою алгоритму Віоли-Джонса, розпізнавання людей HOG і т. д.).

- `opencv_calib3d` - калібрування камери, пошук стерео-відповідності та елементи обробки тривимірних даних.

- `opencv_flann` - бібліотека швидкого пошуку найближчих сусідів (FLANN 1.5) і обгортки OpenCV.

- `opencv_contrib` - супутній код, ще не готовий для застосування.

- `opencv_legacy` - застарілий код, збережений заради зворотної сумісності.

- `opencv_gpu` - прискорення деяких функцій OpenCV за рахунок використання відеокарт, що підтримують CUDA.

При розробці програми мовою C++ потрібно підключити заголовні файли відповідних бібліотек

```
#include "opencv2/imgproc.hpp"
```

```
#include "opencv2/highgui.hpp"
#include "opencv2/videoio.hpp"
#include <modbus.h>
```

Гарною практикою є задання адрес пристроїв та регістрів макросами

```
#define HMI_SLAVE_ADDR 8 //Modbus-адреса
#define POSITION_REG 0 //регістр Modbus з
положенням сервопривода
```

Протокол Modbus передбачає адресацію 16-бітними регістрами, тому для зберігання отриманих/відправлених даних за можливості краще використовувати тип *uint16_t*

```
modbus_t *ctxhmi;
uint16_t *hmiRegisters;
```

При створенні конексту потрібно вказати порт, швидкість та формат передачі.

```
ctxhmi = modbus_new_rtu("/dev/ttyUSB0",
9600, 'N', 8, 1);
if(ctxhmi == NULL) {
    fprintf(stderr, "Unable to create the
libmodbus
context: %s\n",modbus_strerror(errno));
    return -1;
}
```

Таймаут очікування відповіді від веденого пристрою задають з використанням *struct timeval*

```
modbus_set_debug(ctxhmi, TRUE);
modbus_set_slave(ctxhmi,
HMI_SLAVE_ADDR);
struct timeval response_timeout;
response_timeout.tv_sec = 0;
response_timeout.tv_usec = 500000;
modbus_set_response_timeout(ctxhmi,
&response_timeout);
```

Після чого можна виконувати підключення

```
modbus_connect(ctxhmi);
hmiRegisters = (uint16_t *) malloc(7 *
sizeof(uint16_t));
memset(hmiRegisters, 0, 7 *
sizeof(uint16_t));
```

```
sleep(3);
```

Остання затримка на 3 с потрібна, щоб дати завантажувачу Arduino вийти з режиму очікування прошивки.

Після запуску основного циклу обробки відеопотоку кожен кадр, як правило, містить шуми. Оскільки планується відстеження положення порівняно великого об'єкта, вплив шумів можна зменшити згладжуванням

```
blur(frame, frame, Size(10,10));
```

Часто виділення об'єктів у кадрі зручніше робити в колірній моделі HSV. Виокремити об'єкти з насиченістю кольору вище 198 можна так

```
cvtColor(frame, frame_HSV, COLOR_BGR2HSV);
```

```
split( frame_HSV, hsv_planes );
```

```
threshold( hsv_planes[1], dst, 198, 255, THRESH_BINARY );
```

Вивести користувачу знайдені об'єкти можна, показавши їх контури

```
findContours( dst, contours, hierarchy, RETR_TREE, CHAIN_APPROX_SIMPLE );
```

```
for( size_t i = 0; i < contours.size(); i++ )
```

```
{
```

```
    Scalar color = Scalar( 255, 0, 0 );
```

```
    drawContours( frame, contours,
```

```
(int)i, color, 2, LINE_8, hierarchy, 0 );
```

```
}
```

Кожен контур містить масив точок. Наприклад, координату x початкової точки першого контуру можна отримати

```
x=contours[0][0].x;
```

Записати значення в регістр іншого веденого Modbus-пристрою можна функцією

```
modbus_write_register(ctxhmi, POSITION_REG, angle);
```

За умови встановлених бібліотек при використанні компілятора GCC скопіювати програму можна командою

```
g++ -o main main.cpp -I/usr/include/modbus  
`pkg-config opencv --cflags --libs` -lmodbus
```

Програма роботи

1. Використовуючи приклади до бібліотеки

комп'ютерного зору OpenCV, створити програму, яка виводитиме на екран горизонтальну координату об'єкта.

2. Налаштувати зв'язок між комп'ютером і платою Arduino за протоколом Modbus RTU з використанням бібліотеки libmodbus.

3. Об'єднати створені програми, забезпечивши переміщення сервоприводу залежно від положення об'єкта в кадрі.

Порядок виконання роботи

1. Завантажити бібліотеку OpenCV на комп'ютер. Створити програму на базі прикладу https://docs.opencv.org/3.4/da/d97/tutorial_threshold_inRange.html. Програма повинна виділяти об'єкт на кадрі, отриманому з камери або відеофайла, за допомогою порогового фільтра, що застосовується до компоненти H, S або V цього кадру.

2. Пов'язати горизонтальну координату об'єкта з числом 0...180, яке надалі відповідатиме куту повороту сервоприводу, й вивести його поверх кадру.

3. Використовуючи бібліотеку ModbusSlave, розробити програму для Arduino, що вміст першого (нульового) регістра зберігання (адреса 40001 в даній бібліотеці) інтерпретує як кут повороту сервопривода і пересилає відповідний імпульс керування на сервопривод, підключений до 9-го контакту плати Arduino Uno. Адреса пристрою – 8.

```
#include <modbus.h>
#include <modbusDevice.h>
#include <modbusRegBank.h>
#include <modbusSlave.h>
#include <Servo.h>
#define HMI_SLAVE_ADDR 8
modbusDevice regBank;
modbusSlave slave;
Servo s1;
void setup()
{
```



```

    sl.attach(9);
    pinMode(13, OUTPUT);
    regBank.setId(HMI_SLAVE_ADDR);
    regBank.add(40001);
    slave._device = &regBank;
    slave.setBaud(9600);
    regBank.set(40001, 1);    //запис 1 в
    register 40001
}

void loop()
{
    slave.run(); //отримання Modbus-запитів,
    надсилання у відповідь значення регістра або
    його модифікація
    int angle=regBank.get(40001); //зчитуємо
    поточне значення регістра й надсилаємо на
    сервопривод з врахуванням обмежень
    if(angle>180) angle=180;
    sl.write(angle);
}

```

4. Використовуючи бібліотеку `libmodbus` (<https://libmodbus.org/download/>), підключитись за протоколом Modbus RTU та пересвідчитись, що значення, які надсилаються з ПК, Arduino коректно отримує та змінює положення сервоприводу.

5. Додати код програми з `libmodbus` до програми, написаної в п.2 порядку виконання роботи так, щоб змінна, пов'язана з горизонтальною координатою об'єкта, передавалась з використанням протоколу Modbus RTU на Arduino Uno. Запустити отриману програму й переконатись, що при переміщенні об'єкта в полі зору камери або на відео сервопривод змінює своє положення.

Контрольні запитання

1. Яку частину OpenCV потрібно задіяти для виведення відео користувачеві?
2. Яка функція бібліотеки `libmodbus` дозволяє вказати адресу веденого пристрою?

3. Яка функція OpenCV дозволяє виявити ділянки зображення із значеннями пікселя, вищими заданого порогу?

4. Яка функція бібліотеки libmodbus записує значення у вказаний регістр веденого пристрою?

5. Який метод об'єкта modbusSlave повинен викликатись в нескінченному циклі, щоб забезпечити обробку запитів від Master-пристрою?

Інформаційні ресурси

1. Arduino docs | Arduino documentation. URL: <https://docs.arduino.cc/>

2. OpenCV documentation. URL: <https://docs.opencv.org/3.4.16/>

3. Libmodbus documentation. URL: <https://libmodbus.org/documentation/>.

4. Evans B. Arduino programming notebook. URL: <http://engineering.nyu.edu/gk12/amps-cbri/pdf/ArduinoBooks/Arduino%20Programming%20Notebook.pdf>.